



**Università degli
Studi di Salerno**



Giuseppe Sabetta

Sara Gallo

Saverio D'Avanzo

G.Antonio Cetrulo



Indice

Cos'è UniClass?

Perché UniClass?

Problemi e Soluzioni

Competitor

Come funziona?

Architettura del Sistema

Interfaccia Utente

Implementazione

Testing e Validazione

Impatti e Benefici

Roadmap di Sviluppo

Conclusioni

Team



UniClass è un ambiente digitale open-source.

Questo applicativo non si limita a migliorare l'esperienza degli studenti universitari, ma semplifica anche le interazioni dei docenti con il sistema e con gli studenti stessi.

Il nostro ateneo offre numerosi servizi, tra cui diverse piattaforme per migliorare l'esperienza universitaria.

Le funzionalità offerte da questi strumenti sono spesso troppo simili e si sovrappongono tra loro
(ad esempio, per sapere se un'aula è libera o occupata, bisogna usare due applicazioni web differenti su più piattaforme).

Il nostro obiettivo è quindi **unificare**,
raccogliere e **migliorare questi strumenti**
in un **unico ambiente** semplice e intuitivo.





Problemi e soluzioni

Verificare se aula è libera o occupata su più piattaforme

Riunire le funzionalità delle diverse piattaforme in un unico ambiente

Disorganizzazione nella gestione degli orari

Semplificare la visualizzazione e calendarizzazione di questi ultimi

Interazione tra studenti, docenti e coordinatori spesso complicate

Semplificare le interazioni attraverso servizi di messaggistica

[Torna all'indice](#)



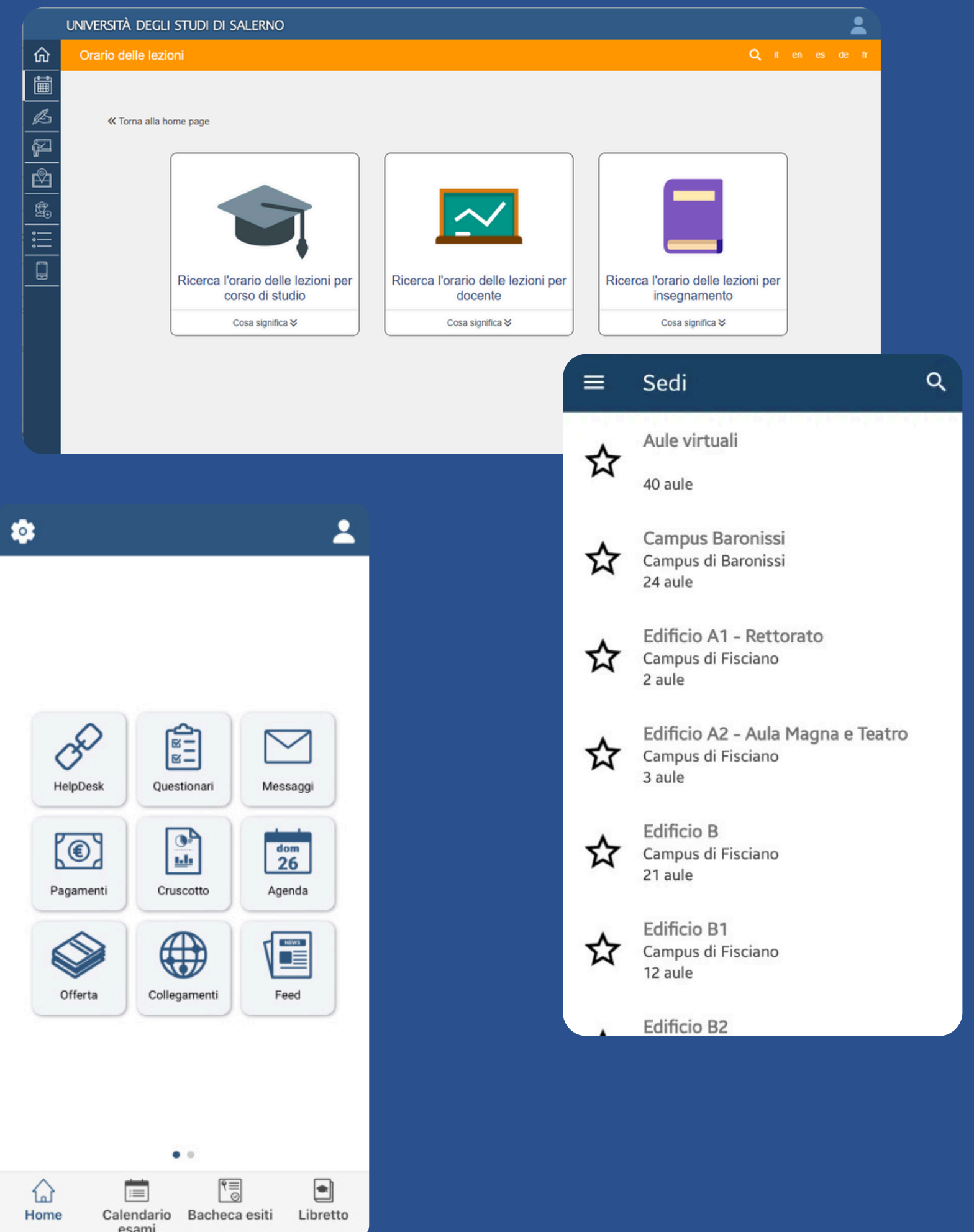
Competitors

Essendo UniClass un'applicazione pensata appositamente per ambienti universitari, i nostri diretti competitor non sono altro che le piattaforme già esistenti messe a disposizione dall'ateneo.

Easy Course

UniSA Lezioni

MyUniSalerno



come funziona UniClass?

Abbiamo iniziato a progettare UniClass identificando le principali funzionalità del sito e mappandole attraverso casi d'uso.

Questo ci ha permesso di definire i requisiti chiave e di concentrarci sulle esigenze degli utenti. Di seguito alcuni dei casi d'uso individuati:



Visualizza Aule

- Attore: Utente
- Descrizione: L'utente accede alla piattaforma per visualizzare le aule libere disponibili in un edificio specifico.
- Flusso: Selezione dell'edificio tramite menu, visualizzazione delle aule e dei relativi orari di disponibilità.



Visualizza Orario

- Attore: Ospite
- Descrizione: L'utente non autenticato può consultare gli orari di un corso specifico.
- Flusso: Compilazione di un form con i dettagli del corso, visualizzazione degli orari.



Messaggistica da parte dello Studente

- Attore: Studente
- Descrizione: Lo studente accede alla piattaforma per inviare un messaggio a un docente tramite il sistema di messaggistica integrato.
- Flusso: Accesso al menù laterale, selezione "Messaggistica", scelta del destinatario, scrittura e invio messaggio, visualizzazione del messaggio inviato nella chat.

Architettura del sistema

Abbiamo ritenuto opportuno sviluppare la nostra piattaforma in modo che l'utente potesse interagire con quest'ultima tramite un'interfaccia web.

Per poter fare ciò, abbiamo utilizzato i seguenti linguaggi e strumenti:



HTML, CSS, JavaScript

Per il lato front-end



Jakarta EE 11, Maven, TomEE, Node.js

Per il lato backend



PostgreSQL, Eclipse Link, JPA

Per gestire la persistenza



IntelliJ IDEA, GitHub

IDE e strumenti utilizzati dal team



UI & UX

L'interfaccia utente di **UniClass** è stata progettata per essere chiara e facile da usare, con un layout *ordinato* che facilita la navigazione.

I colori scelti, una combinazione di **blu** e **bianco**, rendono l'interfaccia visivamente piacevole e aiutano a migliorare la leggibilità dei contenuti.

Nella parte superiore, l'**header** include un'icona menu a sinistra per accedere rapidamente alle opzioni principali e un'icona profilo a destra per il login.

Al centro dell'header si trova il logo UniClass, che rafforza l'identità visiva del sito.

Il **footer**, nella parte inferiore, contiene i crediti e i link alle policy, con un design semplice che non distrae dall'utilizzo della pagina. Nel complesso, l'interfaccia è **funzionale** e **ben organizzata**, pensata per garantire un'esperienza fluida agli utenti.



Implementazione

La nostra piattaforma offre agli utenti diverse funzionalità utili, tra cui il ChatBot.

Questo strumento permette di interagire in modo semplice e veloce, ottenendo risposte adatte alle proprie esigenze.

Il nostro chatbot è stato sviluppato utilizzando un wrapper per Node.js della piattaforma **character.ai**. Questo approccio gli permette di distinguersi dai classici chatbot con domande preimpostate e risposte standard.

Grazie a questa tecnologia, il chatbot è in grado di sostenere conversazioni più naturali e coinvolgenti, offrendo agli utenti un'esperienza interattiva più fluida e immediata.



Oltre al chatbot, il nostro sito offre agli utenti una mappa interattiva del campus, basata sull'API di **Google Maps**.

Grazie a questa integrazione, è possibile esplorare facilmente il campus, trovare percorsi, punti di interesse e ottenere indicazioni in tempo reale.

La mappa è progettata per essere intuitiva e sempre aggiornata, rendendo l'esperienza di navigazione semplice e pratica



Testing e Validazione

[Torna all'indice](#)



Individuazione Casi Di Testing

In fase di documentazione, sono stati individuati i casi di testing all'interno del documento Test Plan e Specification



Testing Unitario

Il testing unitario è una tecnica di verifica del software che si concentra sulla validazione di singole unità di codice, come metodi o classi, per assicurarsi che funzionino come previsto. Nel caso di UniClass questi test sono stati scritti dopo la scrittura del codice



Testing di Integrazione

Il testing di integrazione verifica il funzionamento corretto tra più componenti o moduli del sistema, come: interazioni con database, servizi web e API, servlet e JSP.



Strumenti di Testing

Per il testing Unitario, UniClass si basa sull'utilizzo di JUnit 5 e Mockito.

Per quanto riguarda il testing di integrazione, è stato testato tramite Selenium IDE, un tool di automazione per il testing delle interfacce web.

Vantaggi:

- Individua bug precocemente
- Facilita la manutenzione del codice
- Aumenta la fiducia nelle modifiche e nei refactoring
- Rende il codice più modulare e ben progettato

```
Test Case 1: Login con credenziali corrette|
Email inserita: giacomoporetti@unisa.it
Password inserita: 2222WxY$
Email trovata nel mock DB: giacomoporetti@unisa.it
Utente trovato nel mock DB: giacomoporetti@unisa.it dfac3b52a6ff34d66005efc4ebc3616b36afebfdddec bfe2ae067765c17bf94d3
URL: /UniClass/Home
```

Instantanea del Testing Unitario sulla LoginServlet

Integration Test

[Torna all'indice](#)

Vantaggi

Verifica che la servlet e il database interagiscano correttamente
Assicura che il container JakartaEE (TomcatEE) gestisca i componenti nel modo giusto
Rileva problemi di configurazione prima del rilascio

✓ 	AttivazioneTest (com.example.integration)	23 sec 357 ms
	tC7notPresentBothNotFormattedMatricola	3 sec 499 ms
	tC6notPresentBothAndNotFormattedEmail	2 sec 805 ms
	tC3mailNotPresentDB	3 sec 175 ms
	tC2associazioneErrata	2 sec 824 ms
	tC5notPresentBothInfo	2 sec 740 ms
	tC8nothingGood	2 sec 824 ms
	tC1AllValid	2 sec 727 ms
	tC4emailNotFormatted	2 sec 763 ms

Instantanea dell' Integration Testing su AttivaUtentiServlet

Impatto e benefici di



Fin dall'inizio, il progetto è stato sviluppato con l'obiettivo di integrare le funzionalità *essenziali* di cui ogni studente e docente potesse avere bisogno

UniClass potrebbe essere considerato un progetto già **completo**;

è però un progetto *dinamico* e in costante sviluppo e aggiornamento.

La sua *architettura flessibile* ci permette di implementare rapidamente nuove funzionalità, migliorare quelle esistenti e adattarci alle esigenze degli utenti

Nei prossimi aggiornamenti, vorremmo introdurre nuove funzionalità per arricchire l'esperienza degli studenti, come **l'agenda** personale dello studente, una **biblioteca** digitale, dei **questionari** interattivi per ogni lezione e un **forum** dedicato alla discussione tra studenti.

RoadMap Di Sviluppo



Fase 1 - Analisi e
Pianificazione



Fase 2 - Sviluppo
delle funzionalità
principali



Fase 3 - Testing e
Ottimizzazione



Fase 4 - Rilascio
Beta e
Monitoraggio



Fase 5 - Sviluppi
futuri e
miglioramenti

[Torna all'indice](#)

Fasi del progetto



Fase 1 - Analisi e Pianificazione

Definizione degli obiettivi e delle funzionalità principali

- Identificazione dei bisogni degli studenti e dei docenti.
- Studio delle piattaforme esistenti per individuare gap da colmare.
- Creazione di un elenco di funzionalità chiave (orario, aule libere, chatbot, ecc.).

Scelta delle tecnologie e architettura del sistema

- Backend: JakartaEE 11 con TomcatEE, PostgreSQL per la gestione dei dati.
- Frontend: HTML, CSS, JavaScript con possibili framework.
- Hosting & Server: Tomcat Plume per l'hosting dell'applicazione.

Progettazione dell'architettura del database

- Strutturazione delle tabelle per utenti, orari, aule, messaggi.



Fase 2 - Sviluppo delle funzionalità principali

- **Implementazione della gestione utenti (login e registrazione)**
- **Sistema di ricerca delle aule libere**
- **Chatbot per risposte automatiche**



Fase 3 - Testing e Ottimizzazione

- **Integration e Unit Test**
- **Test di usabilità con studenti e docenti**
- **Sicurezza e protezione dei dati**

Fasi del progetto



Fase 4 - Rilascio Beta e Monitoraggio

Lancio di una versione Beta

- Pubblicazione del sito in ambiente test.
- Apertura agli studenti per testare le funzionalità principali.

Monitoraggio e raccolta feedback

- Analisi dei dati di utilizzo per migliorare il sistema.
- Ottimizzazione delle performance e correzione bug.



Fase 5 - Sviluppi futuri e miglioramenti

Aggiunta di nuove funzionalità basate sul feedback

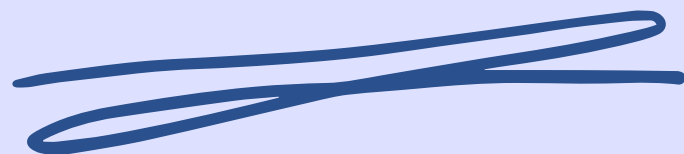
- Notifiche push per aggiornamenti sugli orari e lezioni annullate.
- Supporto per più lingue.
- Integrazione con sistemi universitari esistenti (es. Moodle, Microsoft Teams).
- Agenda Personale dello studente
- Biblioteca Digitale
- Questionari Interattivi

Ottimizzazione mobile

- Creazione di un'app mobile o versione mobile-friendly.

Promozione e diffusione del progetto

- Coinvolgimento di più università per un'eventuale adozione ufficiale.





Utilizzo delle risorse

Per organizzare lo sviluppo di UniClass abbiamo utilizzato Trello, creando una board con le diverse fasi del progetto. Ho suddiviso le attività in liste, assegnando priorità e scadenze, così da mantenere un workflow chiaro ed efficiente."

[Torna all'indice](#)



Grazie per l'attenzione!
Contattaci in caso
di domande.

`g.sabetta3@studenti.unisa.it`

`s.davanzo5@studenti.unisa.it`

`s.gallo52@studenti.unisa.it`

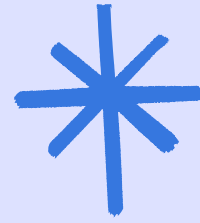
`g.cetrulo4@studenti.unisa.it`

`https://github.com/gssab3/UniClass`



[Torna all'indice](#)

Il team



Sara Gallo

Studentessa



Antonio G. Cetrulo

Studente



Saverio D'Avanzo

Studente



Giuseppe Sabetta

Studente

[Torna all'indice](#)